

COMPUTER NETWORKS PROJECT

NetLab

Network Performance Analyzer & Simulator

A plain-language guide for the team: what we are building, what each phase does, and how to explain it to the professor.

Team	Daksh Arora · Aniket Bhayana · Ishita Anant
Repository	github.com/dakshcodez/netlab
Version	October 2026: Phase 1 built, Phases 2 and 3 planned

What's inside

Section	What you will learn
1. The project in one page	The big idea and why it is stronger than a normal course project
2. The tools, explained simply	What Packet Tracer, Mininet, Wireshark etc. actually are
3. How the work is organised	Phases, branches on GitHub, who merges what
4. Phase 1: Routing	How data finds its way across a network (built)
5. Phase 2: Reliable delivery	How computers catch and fix lost or damaged data (planned)
6. Phase 3: DNS, TCP and the final app	How names become addresses, how traffic speeds up and slows down (planned)
7. Presenting to the professor	A 2-minute pitch, tips, and common questions
8. Glossary	Every technical word in this guide, in one sentence each

How to read this guide

You do not need to read the code to understand the project. Each phase section has the same layout: **the idea in everyday words** → **what we built** → **the experiments** → **what we expect to see** → **how to say it to the professor** → **likely questions with answers**.

Words in the glossary (Section 8) are explained the first time they appear, too.

SECTION 1

The project in one page

The internet looks like magic from the outside: you type a website name and a page appears. Underneath, a handful of clever rules (called **protocols**) work together: one decides *which way* data should travel, one makes sure data *arrives undamaged*, one turns *names into addresses*, and one decides *how fast* to send. Our project takes each of these rules, builds it, and **measures how well it really works**.

The one idea that makes our project stand out

Most student projects show that something *works*. We go further: for every topic we look at it in **three different ways** and put all three on the same graph.

1. Theory

The formula from the textbook.
"On paper, this should take 40 seconds."

2. Simulation

Our own Python program that imitates the protocol step by step. *"Our program says 38 seconds."*

3. Reality

Real networking software running on a virtual network we build. *"We measured 41 seconds."*

When the three agree, we have shown we truly understand the topic. When they *disagree*, that is even more interesting: we explain *why* (for example, real computers have small delays that the textbook formula ignores). Professors value this kind of thinking far more than a demo that just "works".

Everyday analogy

Think of a weather forecast. The **theory** is the meteorology textbook, the **simulation** is the forecast computer, and the **reality** is looking out of the window. A good meteorologist compares all three and explains the gaps. That is exactly what we do, but for computer networks.

What the course syllabus gets covered

Network layer (the "layer")	Topic	Phase
Network layer	IP addresses, subnets, routing (RIP vs OSPF)	1
Data link layer	Error detection (checksum, CRC, Hamming), retransmission (ARQ)	2
Transport layer	TCP: connection setup, congestion control, fairness	3
Application layer	DNS: how website names become IP addresses	3

SECTION 2

The tools, explained simply

We do not build everything from scratch. We use real, industry-standard tools to create the network, and write our own code only where it shows understanding.

Tool	What it is, in plain words	Used in
Cisco Packet Tracer	A drag-and-drop program where you draw a network (computers, routers, cables) and it behaves like real equipment. Great for showing <i>what</i> happens.	Phase 1
Mininet	Creates a whole fake network <i>inside one computer</i> : many virtual computers and routers connected by virtual cables. Unlike Packet Tracer, it runs real Linux networking, so measurements are realistic.	All phases
FRRouting (FRR)	Real router software, the same kind used by internet companies. We run it on Mininet's virtual routers so they talk real RIP and OSPF.	Phase 1
netem ("network emulator")	A Linux feature that makes a cable deliberately bad: slower, delayed, or losing/damaging some data. Lets us test protocols under bad conditions on demand.	All phases
iperf3	A speed test between two computers (like speedtest.net, but inside our network).	1, 3
ping / traceroute	ping checks if a computer answers and how fast. traceroute lists every router a message passes through.	1, 3
Wireshark	A "microscope" that shows every single packet travelling on a cable.	All phases
Python	The language we write our own simulators, experiments and graphs in.	All phases
Ubuntu virtual machine	Mininet and FRR need Linux. Everyone runs the same Ubuntu VM so results are identical on every laptop.	All phases

Why use Mininet if we already have Packet Tracer?

Packet Tracer is a *picture* of a network: perfect for showing messages and configuration, but its timings are not realistic. Mininet runs the *real* software that runs the internet, so when we say "OSPF recovered in 38 seconds", that is a real measurement. We use Packet Tracer to **show** and Mininet to **measure**.

SECTION 3

How the work is organised

The project is split into three phases, one per review. Each team member owns one phase. All code lives in one GitHub repository, and each phase has its own **branch**: a separate copy of the project where that person can work without disturbing others.

Phase	Branch	Review	Theme	Owner
1	phase1	Review 1	Building the test network + routing	_____
2	phase2	Review 2	Error detection + reliable delivery	_____
3	phase3	Review 3	DNS, TCP, analyzer, dashboard + final version	_____

How the branches fit together

- **main** is the clean, official version of the project. Nobody can change it directly. Changes arrive as a **pull request** (a request to merge), and Daksh approves it.
- After Review 1, **phase1** is merged into main. The Phase 2 owner then pulls main into phase2 so they can use the test network built in Phase 1.
- After Review 2, the same happens for phase2 → main → phase3.
- **phase3** ends up containing everything and becomes the final version.

The shared “results format”

Every experiment, from every phase, saves its results in the same way (a JSON file plus a CSV spreadsheet in the `results/` folder). This is what lets the final dashboard in Phase 3 show graphs from all three phases in one place. If you add an experiment, save results with `netlab.common.results`.

Where things are in the repository

Folder	What's in it
netlab/	The Python code, split by topic (routing, testbed, addressing, ...)
experiments/	One script per experiment; run it and it produces results + graphs
docs/	The plan, setup instructions, this guide, and per-phase guides
tests/	Automatic checks that the code gives correct answers
results/	Generated output (graphs, tables). Not stored on GitHub; re-created by running experiments

Routing: how data finds its way

Phase 1 status

Done: all code, the simulator, 18 automatic tests passing, the Packet Tracer guide, the figures above.

To do (Phase 1 owner): build the Packet Tracer file from the guide; set up the Ubuntu VM and run the three real-network experiments; capture RIP/OSPF packets in Wireshark for the report.

The idea in everyday words

When you send a message across the internet, it hops through many **routers**: boxes that read the destination address and decide which cable to send it down next, like junctions on a road trip. Routers learn the roads automatically by chatting with each other using a **routing protocol**. We compare the two classic protocols:

	RIP (Routing Information Protocol)	OSPF (Open Shortest Path First)
How it decides	Counts hops (number of routers). Fewest hops wins.	Adds up a cost based on cable speed. Cheapest (fastest) total wins.
How it learns	Each router only tells its neighbours "here is how far I am from everything". Called distance vector .	Every router learns the <i>whole map</i> and calculates the best route itself (Dijkstra's algorithm). Called link state .
Road-trip analogy	A GPS that only minimises the <i>number of turns</i> , even if one road is a dirt track.	A GPS that knows every road's speed limit and picks the fastest route.

What we built

Piece	In plain words
Address planner (VLSM)	Every device needs an IP address, like a house address. Different departments need different numbers of addresses. Our tool divides a block of addresses efficiently: biggest department first, no wasted space. We used it to plan the Packet Tracer network.
Packet Tracer lab	A small company network: a head office with 3 departments (separated into VLANs, i.e. virtual groups on one switch), a branch office, and automatic address handout (DHCP). We configure static routes, then RIP, then OSPF. Step-by-step guide: docs/phase1/PACKET_TRACER.md.
Routing simulator	Our own Python version of both protocols' algorithms. It predicts which way every router will send data.
The test network (testbed)	Code that builds any network we describe inside Mininet, runs real router software (FRR) on it, and can break cables on purpose. Phases 2 and 3 reuse this.

Experiment 1: the “trap” network

We designed a network specifically so the two protocols **disagree**. Between router r1 and router r4 there is a short but very slow road (1 Mbit/s) and a longer road through r2 and r3 that is 100 times faster.

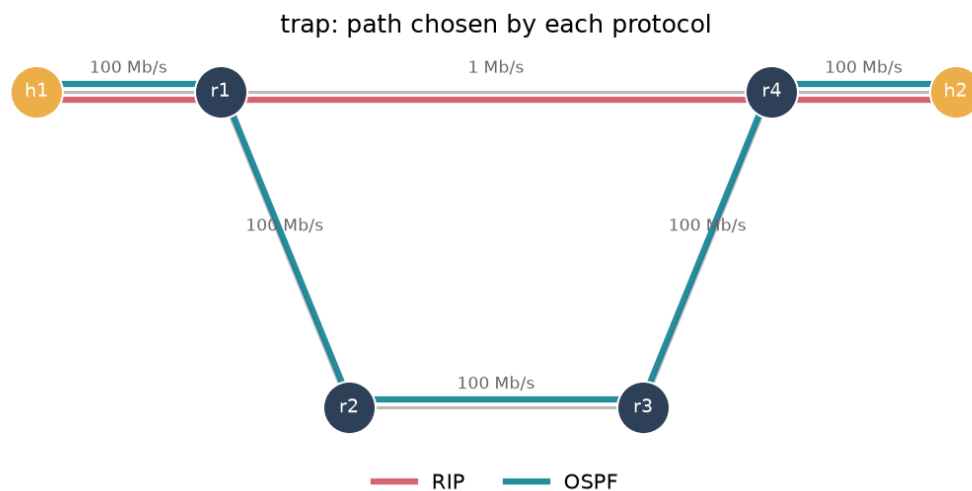


Figure 1: Our simulator's prediction. RIP (red) takes the short, slow link; OSPF (teal) takes the longer, fast route.

We then run real RIP and real OSPF on this network, use **tracert** to see the actual path, and **iperf3** to measure the actual speed. Expected: RIP $\approx$ 1 Mbit/s, OSPF $\approx$ 95 Mbit/s. That's a **$\sim 100\times$ difference** caused purely by how each protocol measures “best”.

Experiment 2: what happens when a cable breaks?

While one computer sends a ping every 10 milliseconds to another, we break a cable on the route and measure how long the connection is dead before routers find a new path. We break the cable in two ways:

- **Unplugged** (“carrier loss”): the router immediately notices the cable is gone, like a road with a big “CLOSED” sign.
- **Silent failure**: the cable looks fine but quietly drops everything, like a road blocked by a landslide around a bend. Routers only notice when they stop hearing from their neighbour for a while.

	Unplugged	Silent failure	Why
OSPF	under ~ 1 second	≈ 30 – 40 seconds	Gives up on a silent neighbour after 40 s (the “dead interval”)
RIP	up to ~ 30 seconds	≈ 150 – 210 seconds	Waits 180 s before forgetting a route, then up to 30 s for the next update

Our code calculates the expected range from the protocols' timer settings (**theory**), and the experiment measures the real outage (**reality**), then plots both together.

Experiment 3: count-to-infinity

RIP has a famous flaw. If a destination becomes unreachable, two routers can keep telling each other “I can get there through you”, each adding 1 to the distance, round after round. They count up slowly until they hit 16, which RIP treats as “infinity / unreachable”. Two fixes exist: **split horizon** (“don't tell a neighbour about a route you learned from them”) and **poison reverse** (“tell them it's unreachable instead”).

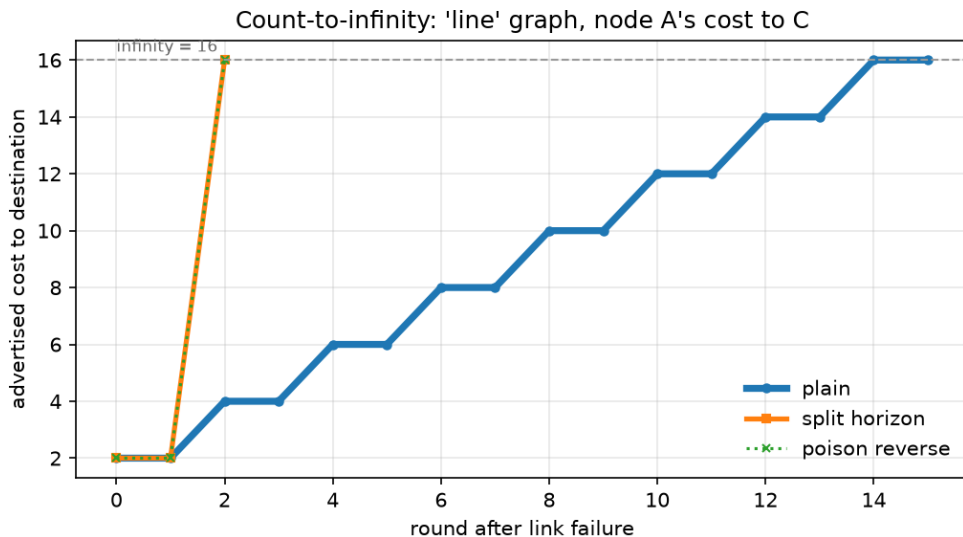


Figure 2: Three routers in a line. Without a fix (blue) the cost climbs to 16 over 15 rounds; with split horizon it is fixed in 2 rounds.

Our interesting finding

Split horizon fixes the problem for three routers in a *line*, but our simulator shows it **still fails for routers in a triangle**: it still takes 15 rounds. The bad information goes around the loop instead of bouncing back. This is why RIP limits infinity to just 16 (so the counting ends quickly), and why modern networks prefer link-state protocols like OSPF, which never have this problem.

Experiment 4: checking our simulator against the real thing

On a bigger 6-router network, our simulator predicts every router's full routing table. We then read the real tables from the real router software and compare them line by line. Goal: **100% match**. This proves our understanding of the algorithms is correct.

How to say it to the professor

“We built a virtual network that runs real router software, and compared RIP and OSPF three ways: theory, our own simulator, and real measurements. We designed a trap network where they choose different paths, and measured a hundred-times speed difference. We also measured how long each takes to recover from a broken cable, and showed that split horizon fixes count-to-infinity in a line but not in a triangle.”

Likely questions

Q: Why does RIP pick the slow link?

A: RIP only counts hops. The direct link is 1 hop; the fast route is 3 hops. RIP has no idea how fast each cable is.

Q: How does OSPF know the speed?

A: Each interface has a cost = reference bandwidth ÷ link bandwidth. We use 1000 Mbit/s as the reference, so a 100 Mbit/s link costs 10 and a 1 Mbit/s link costs 1000. OSPF adds the costs and picks the smallest total.

Q: What is VLSM and why use it?

A: Variable Length Subnet Masking: giving each network a block sized to what it needs instead of the same size for all. A 100-person department gets a /25 (126 addresses), a link between two routers gets a /30 (2 addresses). It avoids wasting addresses.

Q: Why is OSPF faster to recover from a silent failure?

A: OSPF routers say “hello” every 10 s and declare a neighbour dead after 40 s of silence. RIP waits 180 s before it gives up on a route. In real networks, a tool called BFD is added to detect failures in milliseconds.

Q: Why not just use Packet Tracer?

A: Packet Tracer is good for showing configuration and messages, but its timing isn't realistic. Mininet runs real Linux networking and real router software, so our measurements mean something.

Reliable delivery: catching and fixing mistakes

Phase 2 status

Planned. Most of Phase 2 is pure Python and can start immediately on the phase2 branch. Only the “real mode” needs the Phase 1 test network, which becomes available once phase1 is merged into main.

The idea in everyday words

Data travels as 1s and 0s. Along the way, noise can flip some bits (a 1 becomes a 0) and some packets get lost completely. Computers solve this in two steps:

- **Detect damage:** add a small “check value” to every packet. The receiver recalculates it; if it doesn't match, the packet was damaged. Like the check digit on a barcode or an ISBN.
- **Recover:** resend what was lost or damaged. The rules for when and what to resend are called **ARQ** (Automatic Repeat reQuest).

Part A: error detection, built from scratch

Method	How it works (simply)	Strength
Parity bit	Add one bit so the total number of 1s is even.	Catches any single flipped bit; misses two.
2D parity	Arrange data in a grid, add parity to each row and column.	Can even locate (and fix) a single flipped bit.
Internet checksum	Add up the data in 16-bit chunks; send the sum. Used in IP, TCP, UDP.	Fast, but has blind spots, e.g. swapping two chunks gives the same sum.
CRC-32	Treat data as one huge number, divide by a special number, send the remainder. Used in Ethernet and Wi-Fi.	Very strong: guaranteed to catch any burst of errors up to 32 bits long.
Hamming (7,4)	Add 3 clever check bits to every 4 data bits.	Can fix a single wrong bit itself, without resending.

Experiment: damage millions of packets on purpose (random flips, and “bursts” of errors in a row) and count how often each method *fails to notice*. This turns textbook claims into measured numbers, e.g. showing exactly where the checksum is blind and CRC is not.

Part B: ARQ: three strategies for resending

Imagine mailing a 100-page document one page per envelope, through a post office that sometimes loses envelopes:

Strategy	Analogy	Trade-off
Stop-and-Wait	Send page 1, wait for the “got it” reply, then send page 2.	Simple, but very slow on long-distance links: most time is spent waiting.
Go-Back-N	Send several pages without waiting. If page 5 is lost, resend page 5 <i>and every page after it</i> .	Fast when little is lost; wasteful when losses are common.
Selective Repeat	Send several pages; if page 5 is lost, resend <i>only</i> page 5.	Most efficient, but the receiver must store out-of-order pages.

We write each strategy once and run it in two modes: **simulated** (inside Python, perfectly repeatable) and **real** (actual network messages between two virtual computers on the Phase 1 test network, with netem losing and delaying packets). The textbook gives formulas for each strategy's efficiency, so we get the full **theory vs simulation vs reality** comparison.

Experiments and expected results

- **Efficiency vs loss rate:** as more packets get lost, how much useful data gets through?
Expected: Selective Repeat > Go-Back-N > Stop-and-Wait, with the gap growing as loss grows.
- **Window size:** how many packets to send before waiting? Bigger helps, until losses make Go-Back-N waste effort.
- **Timeout setting:** resend too early and you send duplicates; too late and you sit idle.
- **Reordering:** when packets arrive out of order, Go-Back-N throws good packets away; Selective Repeat keeps them.

How to say it to the professor

“We implemented five error-detection methods from scratch and measured how often each misses errors, including the checksum's blind spots and CRC's burst guarantee. Then we implemented Stop-and-Wait, Go-Back-N and Selective Repeat, ran them both in simulation and over a real lossy network, and plotted them against the textbook formulas.”

Likely questions

Q: Why does Ethernet use CRC instead of a checksum?

A: CRC catches far more error patterns, especially bursts of errors in a row, which are common on cables and radio. Our experiment shows this with numbers.

Q: Why is Stop-and-Wait so slow?

A: The sender sits idle while the packet travels and the reply comes back. On a long link, that waiting time is much bigger than the sending time. The formula is $\text{efficiency} = 1 / (1 + 2a)$, where $a = \text{travel time} \div \text{sending time}$.

Q: Why do simulation and reality differ slightly?

A: Real computers have timer inaccuracies, operating-system scheduling delays and queueing that the formulas ignore. Explaining these gaps is part of our analysis.

DNS, TCP, the analyzer and the final app

Phase 3 status

Planned. The DNS message encoder and the analyzer don't need the test network and can start right away on the phase3 branch. The final version of the whole project lives on phase3.

Part A: DNS, the internet's phonebook

Computers find each other using numeric IP addresses (like 142.250.183.14), but people type names (like google.com). **DNS** (Domain Name System) translates names into addresses. It is organised as a hierarchy, like asking for directions in stages:

- Ask a **root server**: “Who handles .com?” → “Ask the .com server.”
- Ask the **.com server**: “Who handles google.com?” → “Ask Google's server.”
- Ask **Google's server**: “What is the address of google.com?” → the answer.
- A **resolver** does this legwork for you and **remembers (caches)** answers for a while, so the next lookup is instant.

What we build: a complete miniature DNS system on our test network: our own root server, a “.lab” top-level server, website servers, and a caching resolver, all written by us, speaking the real DNS message format. Normal tools like dig can query it.

- **Experiment:** first lookup (4 steps) vs repeated lookup (instant from cache), measured and shown in Wireshark.
- **Security demo (the “wow” moment): DNS cache poisoning.** If a resolver uses predictable message IDs, an attacker can sneak in a fake answer, sending users to the wrong website. We show the attack working inside our isolated lab, then the fix (random IDs and ports) making it practically impossible.

Part B: TCP and congestion control, the traffic police

TCP is the protocol behind web pages, downloads and email. Besides resending lost data, it controls *how fast* to send. Too slow wastes the network; too fast causes a traffic jam (**congestion**) where routers drop packets. TCP starts slowly, speeds up, and backs off when it detects loss, producing a famous “sawtooth” speed pattern.

Experiment	What we show
Reno vs CUBIC vs BBR	Linux has several speed-control algorithms. We run each over a congested link and plot how fast they send over time: slow start, the sawtooth, recovery from loss.
Fairness	Two downloads share one link. Do they split it 50/50? We measure with Jain's fairness index. Some algorithm pairs are famously unfair.

Experiment	What we show
Bufferbloat	When routers have huge queues, everything slows down under load, the cause of laggy video calls while someone downloads. Easy to show, real-world problem.
Our own TCP model	A simple simulator of TCP Reno whose graph we lay on top of real Linux TCP: theory vs reality again.

Part C: the analyzer and the dashboard

- **Analyzer:** our own tool that reads a Wireshark capture file and automatically reports what happened: connection setups, round-trip times, resent packets, speed over time, DNS lookups. This is the “Analyzer” in the project name.
- **Dashboard:** a simple web page where you choose an experiment from any phase, move sliders (e.g. “packet loss 5%”), press Run, and watch the graphs appear. Perfect for a live demo at the final review.
- **Final integration:** merge all phases, and make one command (`make all`) re-run every experiment and regenerate every graph.

How to say it to the professor

“We built a complete DNS hierarchy and demonstrated a cache-poisoning attack and its fix. We compared TCP congestion-control algorithms over a congested link, measured fairness and bufferbloat, and built our own packet analyzer. Everything is available in a dashboard where any experiment can be re-run live.”

Likely questions

Q: Why does DNS cache answers?

A: To avoid repeating the whole 4-step lookup every time. Each answer has a TTL (time-to-live) saying how long it can be remembered.

Q: Is the poisoning demo safe/legal?

A: Yes. It runs entirely inside our own isolated virtual network, attacking our own resolver. Nothing touches the real internet.

Q: What is the difference between CUBIC and BBR?

A: CUBIC (Linux default) slows down when it sees packet loss. BBR (by Google) instead measures the link's bandwidth and delay and paces itself to match, so it handles random loss much better, but can be unfair to CUBIC.

Presenting to the professor

The 2-minute pitch (memorise the gist)

"Our project, NetLab, studies how the internet works layer by layer: routing, reliable delivery, DNS and TCP. What makes it different is that we don't just build demos: for every topic we compare three things: the textbook theory, our own simulation, and measurements from a real virtual network running real networking software. When they agree, it proves we understand the topic; when they disagree, we explain why. For example, we designed a network where RIP and OSPF choose different paths and measured a hundred-times speed difference, and we showed that split horizon doesn't fully fix count-to-infinity. Everything can be re-run with one command and explored in a dashboard."

Tips

- **Lead with a graph, not code.** Professors remember the trap-network figure and the theory-vs-reality plots.
- **Always say which of the three levels you are showing:** "this is the formula", "this is our simulator", "this is the real measurement".
- **Mismatches are good news** if you can explain them. Never hide them.
- **Know your numbers:** RIP infinity = 16; OSPF hello 10 s / dead 40 s; RIP update 30 s / timeout 180 s; CRC-32 catches all bursts ≤ 32 bits; Stop-and-Wait efficiency = $1/(1+2a)$.
- **If you don't know an answer,** say how you would find out using our setup ("we could measure that by changing the netem loss and re-running"). That shows real understanding.
- **Every member should be able to explain all three phases at the one-page level:** this guide is enough for that.

General questions any of us might get

Q: Why did you use existing tools instead of building everything?

A: Because the goal is to understand and measure the protocols, not to rebuild cables. We use real tools to create a realistic network and write our own code exactly where it demonstrates understanding: the algorithms, error detection, ARQ, DNS servers, the analyzer.

Q: How do you know your results are correct?

A: Three ways: automatic tests on our code, comparing our simulator with real software (e.g. 100% routing-table match), and comparing both with textbook formulas.

Q: Can you reproduce your results?

A: Yes. Every experiment is a script; one command re-runs everything and regenerates every graph, and everyone uses the same Ubuntu VM.

SECTION 8

Glossary

Term	Meaning
ARQ	Automatic Repeat reQuest: rules for resending lost or damaged data.
Bandwidth	How much data a link can carry per second, e.g. 100 Mbit/s.
BBR / CUBIC / Reno	Different TCP algorithms for deciding how fast to send.
Branch (Git)	A separate line of work in the repository, so people don't overwrite each other.
Bufferbloat	Excess delay caused by routers holding too many waiting packets in a queue.
Cache	A short-term memory of recent answers, to avoid repeating work.
Checksum	A small value computed from data, used to detect damage.
Congestion	A traffic jam in the network: more data than links can carry.
Convergence	The moment all routers agree on the routes again after a change.
Count-to-infinity	A distance-vector problem where routers slowly count up the distance to a dead destination.
CRC	Cyclic Redundancy Check: a strong error-detection method used by Ethernet and Wi-Fi.
DHCP	Automatically hands out IP addresses to devices when they join a network.
Dijkstra's algorithm	The method OSPF uses to find the cheapest path on a map.
Distance vector	Routing where routers share only distances with neighbours (RIP).
DNS	Domain Name System: translates names like google.com into IP addresses.
FRR	FRRouting: real, open-source router software.
Hamming code	Extra bits that let the receiver fix a single wrong bit by itself.
Hop	One router-to-router step along a path.
IP address	A device's address on a network, e.g. 10.0.0.1.
Latency / delay	How long data takes to travel from one end to the other.
Link state	Routing where every router knows the whole map (OSPF).
Mininet	Software that creates a virtual network of computers and routers inside one machine.
netem	A Linux feature that makes links slower, delayed or lossy on purpose.
OSPF	Open Shortest Path First: link-state routing protocol that picks the cheapest (fastest) route.

Term	Meaning
Packet	A small chunk of data sent across a network, with addresses attached.
Packet loss	When a packet never arrives.
Poison reverse	Telling a neighbour a route is unreachable if you learned it from them.
Protocol	An agreed set of rules for how computers communicate.
Pull request	A request to merge one branch's changes into another, reviewed before merging.
RIP	Routing Information Protocol: distance-vector protocol that picks the fewest hops.
Router	A device that forwards packets towards their destination.
Split horizon	Not telling a neighbour about a route you learned from that neighbour.
Subnet / mask	A block of IP addresses; the mask (like /24) says how big the block is.
TCP	Transmission Control Protocol: reliable delivery plus speed control, used by the web.
Testbed	Our virtual network in Mininet, where real experiments run.
Throughput	How much useful data actually got through per second.
TTL	Time-to-live: how long a cached DNS answer may be kept.
UDP	A simple, fast protocol with no built-in reliability; we add our own ARQ on top.
VLAN	A virtual group that splits one physical switch into separate networks.
VLSM	Giving each subnet a block sized to its needs, to avoid wasting addresses.
Wireshark	A tool that shows every packet on a network, for inspection.